The Scale actor wire protocol

Status: W0 landed, W1 partial, W2/S1 and S2 landed (D154, D155, D159); the frozen subset is conformance-tested; W3–W7 planned. This is the canonical plan. `Notes/binary-format.md` is the early estimation table that seeded it; it is kept as the raw working note.

The decision

Scale implements its own actor wire protocol. It does not depend on the Cap'n Proto runtime, code generator, or library. The design is Cap'n Proto-inspired: segments, struct/list/far pointers, defaults, and a capability table.

It takes the three major Cap'n Proto implementations as references:

1. **C++**, the normative wire encoding and the complete RPC state machine.
2. **Rust**, validation during traversal, explicit `Result` on fallible getters, and lifetime-parameterized reader and builder views.
3. **Go**, grow-on-write buffers, pooling, transport-agnostic framing, and a lightweight connection-concurrency model.

Protobuf is the reference for the code-generator contract and schema evolution; OCapN is the reference for capability references and interop. Neither is a dependency.

This is the **Scale actor wire protocol**, not Cap'n Proto. We do not claim wire compatibility. If compatibility becomes a goal, it is a separate decision with a conformance suite against the reference implementation; any adjustment then needs a version.

Scope

The first protocol is the actor IPC envelope and portable payloads:

1. a segment table, a bounded message, and far pointers;
2. struct, list, text, data, and scalar fields, including defaults;
3. a capability table for actor handles;
4. a version and capability handshake;
5. a length-prefixed frame over a transport, with a Unix socket first.

The first protocol includes a small schema compiler and a Protobuf-style code-generation plugin. It is not an RPC system with promise pipelining, three-party handoff, distributed GC, or an arbitrary object graph. Those wait for a consumer.

Envelope and payload

Keep one message. The root struct is the envelope: target handle, sender handle, optional reply channel, message identity, capability-table reference, and a body pointer. The body is a union or pointer in the same message.

The split is logical, not physical. Routing, authentication, and capability checks read the envelope before the body is traversed. A large tensor or blob may live out-of-line in a separate segment or a chunked stream with a descriptor; that is bulk transport, not a second envelope. There is no second frame.

Protobuf tooling over a Cap'n Proto wire

The schema layer takes Protobuf's tooling and evolution discipline:

1. a descriptor set and a `CodeGeneratorRequest`/`CodeGeneratorResponse` plugin contract, so a new language backend is a plugin rather than a fork;
2. field numbers and reserved ranges as schema metadata, with a compatibility matrix;

3. `oneof` mapped to a Cap'n Proto union.

The wire layer stays Cap'n Proto: layout, ordinals, pointers, segment table, and defaults. Field numbers and reserved ranges never appear on the wire; the compiler maps them to ordinals. `oneof` is a union, not a Protobuf tag.

Two Protobuf features do not map cleanly:

1. **Unknown fields.** Cap'n Proto does not preserve arbitrary unknown fields by tag. Preserve them only with an explicit raw-body policy, and state the policy per schema; otherwise document that unknown fields are dropped.
2. **`Any` and extensions.** Use a typed union plus a schema registry. `AnyPointer` is allowed only behind validation and a whitelist. An open `Any` is not the actor contract.

A codec tag that carries Protobuf wire messages beside Cap'n Proto messages is a second protocol, not a compatibility feature. If both are needed, say so and test both; do not call the pair one wire format.

Compatibility boundary

The frozen subset is Cap'n Proto-compatible in layout and encoding. It now passes the gate: the reference implementation decodes our bytes, our decoder accepts its bytes, and the F# encoder matches the reference bytes for every checked-in vector. The claim is limited to that subset. Adjustments that change the wire are versioned and are not claimed as compatibility.

Golden vectors and reference interop

The frozen subset is pinned by `components/ipc/vectors/`:

1. `scale_actor.capnp` is the schema: `ActorEnvelope` with seven `UInt64` data fields and one `Text` pointer. The field order is the wire order.
2. `<name>.txt` is the Cap'n Proto text form of one message.
3. `<name>.bin` is the framed bytes produced by `capnp encode` from that text. The binary is generated, never hand-edited.

The corpus is: `number` (send 4.5), `text` (down "down normal"), `monitor`, `monitor-ack`, `reply`, and `defaults` (an all-zero numeric send). A malformed corpus covers a truncated frame and a non-struct root pointer.

The interop contract is two-way:

1. The F# `Wire.decode` must decode every `.bin` to the fields in the test table.
2. `components/ipc/vectors/check.sh` regenerates each `.txt` with `capnp encode` and compares the bytes, then decodes the Elixir encoder's bytes with `capnp decode` and checks the fields.

The compatibility claim applies only to this subset. A multi-segment message, a far pointer, a different field order, or a schema change is a new version, not a silent adjustment. `capnp` is the reference tool; the `nix-shell -p capnproto` form is acceptable where it is not installed.

What we take

Component	Reference	What we take and what we adapt
Wire encoding	C++	The normative segment, struct/list pointer, far pointer, and default rules. The specification is authoritative; C++ resolves ambiguity. Do not codify implementation quirks.

Component	Reference	What we take and what we adapt
Pointer validation	Rust	Validate during traversal. Bound the offset, depth, and cycles. Return an explicit failure on every fallible getter; no silent fallback.
Builder and arena	Rust, Go	Exclusive builder access from Rust; grow-on-write and pooling from Go. Avoid C++ manual lifetime discipline.
Borrowed views	Rust	Lifetime-parameterized reader and builder views, kept small enough for the actor API.
Code-generator contract	Protobuf	The `CodeGeneratorRequest` and `CodeGeneratorResponse` stdin/stdout contract, plus a descriptor set. A new language is a plugin, not a fork. Avoid C++ bootstrapping coupling.
Schema evolution	Protobuf, Cap'n Proto	Permissive add/remove-field rules, field numbers, reserved ranges, and a compatibility matrix as schema metadata, with Cap'n Proto's stricter ordinals on the wire. Design for many simultaneous peer versions, not only client-server.
Unknown fields	Protobuf, Cap'n Proto	Protobuf preserves unknown fields by tag; Cap'n Proto does not. Preserve only with an explicit raw-body policy, and document the policy per schema.
Extensions and `Any`	Protobuf, Cap'n Proto	A typed union plus a schema registry. `AnyPointer` only behind validation and a whitelist; an open `Any` is not the actor contract.
RPC state machine	C++	Question/answer tables and promise pipelining as the reference. Deferred; do not copy C++ threading assumptions.
Capability table	C++, Go	Table indices into a runtime-controlled table and per-connection export/import tables. Avoid `AnyPointer` workarounds.
Three-party handoff	Go experimental	The simplest handoff as a working demonstration. Full Level 3 is unsolved; deferred.
Embargoes and E-order	C++	Ordering across promise resolution and proxying. Deferred with the RPC state machine.
Connection concurrency	Go	Dedicated read/write loops, connection state under a narrow lock, no lock on the message-handling hot path.
Framing and transport	Go	Transport-agnostic framing over TCP, WebSocket, or a Unix socket. The actor layer stays above framing.
Distributed GC	C++	Reference-counted export tables and acyclic distributed GC as the reference. Deferred; linear capabilities aim at deterministic release.
Sturdy references	OCapN	Unguessable, re-connectable references and the netlayer abstraction. Do not invent a second scheme.

Component	Reference	What we take and what we adapt
OCapN interop	OCapN	A convergence path and interop tests. Do not commit before OCapN 1.0.
Linear capability safety	OCapN and object-capability literature	Design from first principles. No Cap'n Proto binding implements it.

Staging

1. **W0 — freeze the subset (landed, D159).** The envelope and body schema, the checked-in Cap'n Proto vector corpus, and the two-way reference interop exist. A far pointer remains out of the frozen subset.
2. **W1 — the F# codec (landed for the first subset).** Explicit readers and writers, no reflection, NativeAOT-safe, with numeric and text bodies. The generated backend remains W5.
3. **W2 — two-instance loopback (landed, D154/D155).** Two instances over a Unix socket exchange an envelope, a numeric reply, a monitor acknowledgement, and a down message. This is actor IPC S1 and the first S2 slice.
4. **W3 — capabilities (partial).** A capability table names a non-root worker; the root monitor reports the worker's normal exit, and an unreachable instance is a named outcome. Discovery, authentication, and a capability security handshake remain.
5. **W4 — schema compiler and descriptor set.** A Cap'n Proto IDL subset, a descriptor set, field-number and reserved-range metadata, and the compatibility matrix. Field numbers stay off the wire.
6. **W5 — plugin contract and F# backend.** The `CodeGeneratorRequest`/`CodeGeneratorResponse` contract and an F# backend; generated output is compared with the hand-written codec.
7. **W6 — cross-language adapters (partial).** The Elixir wire harness (`adapters/elixir`, D156) consumes the same envelope for the peer and listener directions. It is hand-written, not plugin-generated; Julia is not started.
8. **W7 — RPC subset.** Question/answer and pipelining only when a consumer needs them; three-party handoff and distributed GC later.

Testing

1. Golden vectors for every encoding feature in W0, checked in and shared by every implementation.
2. A property that a generated value round-trips through the codec.
3. A malformed-input corpus: truncated segment, out-of-bounds pointer, bad far pointer, cycle, excessive depth, invalid capability index, and a message over the size limit.
4. A descriptor and plugin test: a descriptor round-trip, a known backend input and output, and the field-number/reserved-range to ordinal mapping.
5. A Cap'n Proto interop test: the reference implementation decodes our golden
6. vectors and our decoder accepts its bytes. This test gates the compatibility
7. claim.
8. An unknown-field policy test per schema: preserve or drop, never silently reinterpret.
9. A supervised round-trip: monitor a worker capability, send a number, and receive both the reply and the down reason.
10. A cross-language conformance run once a second adapter exists.
11. A NativeAOT check: no reflection, no runtime code generation, no dynamic
12. serialization.

13. A fuzz pass on the decoder before the protocol leaves the lab.

The decoder never accepts a malformed message because a getter was convenient. Every failure is a named value. A pointer that cannot be validated is refused, not repaired.

Risks and opinion

The encoder is the small part. The cost is the ecosystem: a schema compiler, code generation for three host languages, evolution rules, an RPC state machine, three-party handoff, and distributed GC. This plan takes the schema compiler and plugin at W4–W5 and keeps the RPC system and distributed GC out until a consumer needs them.

The three references divide cleanly. C++ is the encoding oracle; Rust is the security model; Go is the operational model. Protobuf and OCapN are secondary references, not dependencies. Taking all three major implementations as inspiration is sound as long as we do not inherit their scope by accident.

The largest risk is cross-language drift. A shared schema and the W0 golden vectors must exist before the first adapter. The second largest risk is claiming Cap'n Proto compatibility before the reference test passes. Compatibility is a tested property, not a label; the protocol has its own name and version for that reason.

The Protobuf tooling is safe only while it stays off the wire. A descriptor set, a plugin contract, field numbers, and reserved ranges are build-time metadata. A field-number tag, a TLV frame, or Protobuf unknown-field preservation on the wire breaks Cap'n Proto compatibility. Unknown fields are the sharp edge: decide per schema whether to preserve raw bytes or drop them, and test the choice.

If the near-term goal is only two instances exchanging actor messages, the segment and pointer machinery is more than the envelope needs. It earns its place if the ultimate goal is zero-copy views, capability tables, and RPC. If that is the goal, this is the right first step; if it is not, start with a smaller fixed envelope and add the pointer format later.

Relationship to other documents

`docs/actor-ipc.typ` is the actor boundary and the S0–S4 staging. `docs/specification.typ` section 11.7 is the normative summary. D152 records the in-repo wire decision; D153 records the Protobuf-tooling-over-Cap'n-Proto-wire refinement and the envelope/body rule. D151 records the actor IPC direction and the rejection of the unified Julia/Elixir language. `docs/decision-records.typ` D109 is the existing `scale_transport_send` seam.